

BUILDING BACKEND SYSTEMS

Week One

How the Internet Talks to Your Code

Seven days. Sockets, HTTP, REST, Pydantic, and layered architecture — built from first principles.

WEEK One of Six	DAYS 1 – 7	STACK Python · FastAPI
---------------------------	----------------------	----------------------------------

Contents

Day 1	What Is a Server, Really? <i>Sockets, ports, localhost, the raw TCP layer</i>
Day 2	HTTP: The Language of the Web <i>Request/response structure, headers, the text protocol</i>
Day 3	HTTP Methods and Status Codes <i>Semantics, idempotency, the codes every engineer must know</i>
Day 4	REST API Design <i>Resources, URL conventions, versioning, pagination</i>
Day 5	JSON and Pydantic <i>Serialisation, validation, nested models, custom errors</i>
Day 6	Application Architecture <i>Controllers, services, repositories — layered separation</i>
Day 7 — Summary	Week One Consolidation <i>Review, reflection, and a clean GitHub push</i>

HOW TO USE THIS DOCUMENT

Each day is 90 minutes: 20 minutes of theory, 60 minutes of practical work, and 10 minutes of reflection. Read the theory before opening your editor. Type every code sample by hand — do not copy and paste. Answer the reflection questions before moving to the next day.

Day 7 is a summary day. No new concept is introduced. Use it to write your weekly summary, review your code, and push a clean repository to GitHub. The habit of weekly consolidation is as important as the daily learning.

The test that runs every day

After every session, ask yourself one question:

If the framework I used today disappeared tomorrow, would I still understand what I built?

If the answer is yes, you learned backend engineering. If the answer is no, go deeper on the concept before moving on.

Day 1

What Is a Server, Really?

Week One · How the Internet Talks to Your Code

Most backend courses hand you a framework on the first day and tell you to run it. You do, it works, and you have no idea why. That is not a problem until something breaks. And it will break. When it does, you need a mental model to debug with, not just a list of commands that worked once.

Today you are going to build the simplest possible server using nothing but Python's raw socket library. No FastAPI. No Django. No framework of any kind. Just Python talking directly to the operating system, which talks to the network. When you finish, you will understand exactly what a framework like FastAPI is doing for you — and that understanding will stay with you for the rest of your career.

THEORY · 20 MINUTES

What a server actually is

The word server gets used for two different things and the confusion causes early frustration. It refers to a physical or virtual machine, and it refers to a program running on that machine. In backend engineering, we almost always mean the program.

A server program does one thing at its core: it sits and waits for incoming connections. It is not actively doing anything until something connects to it.

The moment something connects, the server wakes up, processes whatever was sent, sends something back, and returns to waiting. Every backend system you will build for the next 46 days is an elaboration of that loop.

What a port is and why it exists

Your computer has one IP address but can run dozens of programs simultaneously that all need to receive connections. How does the operating system know which incoming message belongs to which program? Port numbers.

A port is a number between 0 and 65535. When a server program starts, it claims a port number from the operating system: any message arriving on this port belongs to me. The OS keeps a table of which program owns which port and routes messages accordingly. PostgreSQL claims 5432 by default. FastAPI's development server uses 8000. A standard web server listens on 80 for HTTP and 443 for HTTPS. All of these run simultaneously on the same machine because each claims a different number.

What localhost means

Every computer has a special IP address — 127.0.0.1 — that means this machine itself. The hostname localhost is a human-readable alias for that address. When you start a development server and open `http://localhost:8000` in your browser, you are making a real network request that travels from your browser, through the network stack, and back to a server running on the same machine. It never leaves your computer.

This is why your development server is not accessible to anyone else on the internet by default. It is bound to 127.0.0.1, a loopback address that only works locally. In production, a server binds to 0.0.0.0 instead, which means accept connections from any network interface.

What a socket is

A socket is the object in your program that represents one end of a network connection. Think of it as a telephone handset. To receive calls: you pick up a handset, sit by it, wait for it to ring, and when it rings you answer. In code:

you create a socket, bind it to a port, call `listen()`, and then call `accept()` — which pauses your program until a connection arrives, exactly as you would wait by a ringing phone.

Every network communication you will ever work with is built on sockets underneath — HTTP requests, database connections, WebSocket messages. FastAPI, SQLAlchemy, the Redis client — all of them use sockets. You have just never seen them directly because every framework hides them. Today you will not use a framework.

Term	Meaning
<i>Server</i>	A program that listens for incoming network connections and responds to them.
<i>Client</i>	Anything that initiates a connection — a browser, Postman, another server, a script.
<i>Port</i>	A number (0–65535) the OS uses to route incoming messages to the correct program.
<i>localhost</i>	The hostname for 127.0.0.1 — the current machine. Requests to localhost never leave your computer.
<i>Socket</i>	A programming object representing one end of a network connection.
<i>Bind</i>	A server claiming ownership of a port. After binding, messages on that port go to that program.
<i>Listen</i>	The state a server enters after binding — waiting for an incoming connection.
<i>Accept</i>	The act of picking up an incoming connection, as you would answer a ringing phone.

PRACTICAL · 60 MINUTES

You are going to build a raw TCP server in Python using only the socket library. It will listen on a port, accept a connection, receive whatever the client sends, and send a response back. This is the skeleton that every server that has ever existed is built on.

1. Create your project

Create a folder called `building-backend-systems`. This is your GitHub repository for all 46 days. Initialise git and create today's file.

```
mkdir building-backend-systems
cd building-backend-systems
git init
mkdir week-01 && touch week-01/day01 server.py
```

Run in your terminal. Windows users: use Git Bash or PowerShell.

2. Write the raw socket server

Type the following by hand. Do not copy and paste. Typing it forces you to read every line, and reading every line is the point. Every comment is part of the lesson.

```
import socket

# AF_INET = use IPv4 addresses (e.g. 127.0.0.1)
# SOCK_STREAM = TCP: reliable, ordered, connection-based
server_socket = socket.socket(socket.AF_INET,
                               socket.SOCK_STREAM)

# SO_REUSEADDR lets us restart without waiting for the OS to
# release the port - saves frustration during development
server_socket.setsockopt(socket.SOL_SOCKET,
                          socket.SO_REUSEADDR, 1)

# Claim port 9000 on this machine only
server_socket.bind(('127.0.0.1', 9000))

# Begin listening - queue at most 1 waiting connection
server_socket.listen(1)
print('Listening on port 9000...')

while True: # a real server never stops on its own

    # accept() blocks here - program pauses until a client
    # connects
    client_socket, address = server_socket.accept()
    print(f'Connection from {address}')

    # recv(1024): read up to 1024 bytes from the client
    data = client_socket.recv(1024)
    print(f'Received: {data.decode()}')
```



```
# Sockets send bytes, not strings — note the b prefix
client_socket.sendall(b'Hello from your raw socket
server!')

# Close this connection and return to waiting
client_socket.close()
```

week-01/day01_server.py — type this in full before running it

3. Run and connect

Open a terminal and start your server. Open a second terminal and send a request using curl.

```
# Terminal 1
python week-01/day01_server.py

# Terminal 2
curl http://127.0.0.1:9000
```

Watch Terminal 1 — you will see the connection address and the bytes curl sent.

curl will likely show a broken pipe or an empty response. That is expected — your server is not speaking HTTP yet, it is speaking raw TCP. The important thing is that the connection happened and you can see it in your server's output.

What just happened

You wrote a program that the OS gave a port to.

That program blocked at `accept()` until curl connected.

When the connection arrived, your program read bytes, sent bytes back, and closed the connection.

This loop — listen, accept, read, respond, close — is the foundation of every backend system you will ever build.

4. Break things intentionally

These experiments will teach you more than re-reading the theory three times. Work through each one and observe what happens before seeking any explanation.

1. Connect to port 9001 instead of 9000. Read the error message carefully. What is the OS telling you?
2. While your server is running, open a third terminal and try to start a second copy. What error appears and why?
3. Remove the while True loop so the program exits after one connection. Restart it and connect twice. What happens on the second attempt?
4. Change '127.0.0.1' to '0.0.0.0' and restart. Open your browser and visit <http://localhost:9000>. What does binding to 0.0.0.0 change?
5. Add `print(len(data))` after the `recv` call. Connect with a long curl request. What do you observe about bytes received in a single call?

5. Compare with FastAPI (if time remains)

Create a second file and write the simplest possible FastAPI server. The comparison is the lesson.

```
from fastapi import FastAPI

app = FastAPI()

@app.get('/')
def root(): return {'message': 'Hello'}
```

week-01/day01_fastapi.py — run with: `uvicorn day01_fastapi:app --reload`

FastAPI also binds to a port, accepts connections, reads bytes, and sends bytes back. It is doing exactly what your socket server does — plus HTTP parsing, request routing, response serialisation, and header management. The fundamentals are identical. The framework is convenience built on top of those fundamentals.

REFLECTION · 10 MINUTES

Do not skip this section. The goal is to find the edges of your understanding — the places where an explanation feels thin. Those edges are exactly where tomorrow's learning begins.

Question 1

Why does your server need a port number? What would the OS do with an incoming network message if no program had claimed that port? Try to find out — connect to a port with nothing listening and read the error carefully.

Question 2

You tried to start two copies of your server and got an error. What does that error mean at the OS level? What would need to be true for two programs to share a port?

Question 3

Your raw socket server and FastAPI do the same fundamental job. Write one sentence describing that job in plain English — no technical terms.

Question 4

What is the difference between binding to 127.0.0.1 and binding to 0.0.0.0? When would each be appropriate and what are the security implications?

BEFORE TOMORROW

Today you built a server at the level of the operating system. Tomorrow you add a layer: HTTP. HTTP is not magic — it is a specific format for the bytes your server already knows how to send and receive. Before tomorrow's session, run this command and read every line of the output carefully.

```
curl -v http://httpbin.org/get
```

The -v flag shows the full HTTP conversation. Tomorrow every line of it will make sense.

Commit before you close your laptop

git add .

`git commit -m "Day 1: raw TCP socket server, port binding experiments"`

Push to your remote repository on GitHub.

Day 2

HTTP: The Language of the Web

Week One · How the Internet Talks to Your Code

Yesterday you built a server that speaks raw TCP. It can receive bytes and send bytes back. That is enough to build a working server — but the web does not communicate in raw bytes. It communicates in HTTP, a specific format for those bytes. Today you will read HTTP as plain text, understand every field in a request and response, and build a server that actually parses and speaks it.

Most developers use frameworks that handle HTTP automatically and never look at it directly. That is a gap worth closing. When you know what an HTTP request looks like as text, error messages make sense, debugging becomes faster, and nothing about web communication feels mysterious again.

THEORY · 20 MINUTES

HTTP is a text protocol

HTTP is not a binary format. It is plain text with a very specific structure. Every HTTP message — whether a request from a client or a response from a server — follows the same form: a start line, followed by headers, followed by a blank line, followed optionally by a body.

That blank line is critical. It is the separator between the headers and the body. A server reading an HTTP message reads line by line until it sees a blank line, then knows that everything after it is the body. If you were building an HTTP parser from scratch, that blank line is the hinge your parser turns on.

The anatomy of an HTTP request

A request has three parts: the request line, the headers, and the body.

```
POST /students HTTP/1.1
Host: api.myapp.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Length: 47

{"name": "Julien", "course": "Computer Science"}
```

A complete HTTP request as it travels over the wire — plain text

The request line has three pieces: the method (POST), the path (/students), and the HTTP version (HTTP/1.1). That is all the server needs to know what kind of action the client wants and where to perform it.

The headers are key-value pairs separated by a colon. Each one tells the server something about the request — who is sending it, what format the body is in, how long the body is, what credentials the sender has. The Host header is mandatory in HTTP/1.1 because one server can host multiple websites on the same IP address, and the Host header tells the server which site the request is for.

The blank line signals the end of headers. Everything after it is the body. The server knows how many bytes to read because the Content-Length header told it.

The anatomy of an HTTP response

A response follows the same structure: a status line, headers, a blank line, and a body.

```

HTTP/1.1 201 Created
Content-Type: application/json
Content-Length: 62
Location: /students/42

{"id": 42, "name": "Julien", "course": "Computer Science"}

```

A complete HTTP response — the server's reply to the POST request above

The status line has the HTTP version, a numeric status code, and a human-readable reason phrase. The Location header tells the client where to find the newly created resource.

Key headers you must know by name

Term	Meaning
<i>Content-Type</i>	What format the body is in: application/json, text/html, multipart/form-data.
<i>Content-Length</i>	How many bytes are in the body. The receiver uses this to know when the body ends.
<i>Authorization</i>	Carries credentials — most commonly a Bearer token (JWT).
<i>Accept</i>	What format the client can understand in the response.
<i>Host</i>	Which website the request is for. Mandatory in HTTP/1.1.
<i>Location</i>	Included in 201 Created and redirect responses. Points to the relevant resource.
<i>Cache-Control</i>	Instructions about whether and how the response can be cached.
<i>Set-Cookie</i>	Tells the client to store a cookie and send it back on future requests.

PRACTICAL · 60 MINUTES

Today has three parts: reading real HTTP traffic, upgrading your raw socket server to speak HTTP, and inspecting what FastAPI does automatically.

1. Read real HTTP traffic with curl (10 minutes)

Run these commands and read every line of the output. Lines starting with > are the request. Lines starting with < are the response.

```
curl -v http://httpbin.org/get

# Then try a POST
curl -v -X POST http://httpbin.org/post \
  -H 'Content-Type: application/json' \
  -d '{"name": "Julien"}'
```

Identify: the request line, every header, the blank line separator, the status line, the response body.

2. Upgrade your socket server to speak HTTP (35 minutes)

Teach your Day 1 server to read an HTTP request and send a valid HTTP response. Create a new file.

```
import socket

def parse_request(raw: bytes) -> dict:
    text = raw.decode('utf-8', errors='replace')
    lines = text.split('\r\n')
    parts = lines[0].split(' ')
    return {'method': parts[0] if len(parts) > 0 else '',
            'path': parts[1] if len(parts) > 1 else '/'}

def make_response(status: int, body: str) -> bytes:
    reason = {200:'OK', 404:'Not Found', 405:'Method Not Allowed'}.get(status, 'Unknown')
    return (
        f'HTTP/1.1 {status} {reason}\r\n'
        f'Content-Type: application/json\r\n'
        f'Content-Length: {len(body.encode())}\r\n'
        f'Connection: close\r\n'
        f'\r\n'
        f'{body}'
    ).encode()

ROUTES = {
    ('GET', '/'): (200, '{"message": "Welcome"}'),
    ('GET', '/health'): (200, '{"status": "ok"}'),
    ('POST', '/echo'): (200, '{"echo": "received"}')
}
```

```

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(('127.0.0.1', 9000))
server.listen(5)
print('HTTP server listening on http://127.0.0.1:9000')

while True:
    client, addr = server.accept()
    raw = client.recv(4096)
    req = parse_request(raw)
    key = (req['method'], req['path'])
    status, body_str = ROUTES.get(key, (404, '{"error": "Not found"}'))
    client.sendall(make_response(status, body_str))
    client.close()

```

week-01/day02_http_server.py — run it and test with curl, then open it in your browser

3. Compare with FastAPI (15 minutes)

Build matching FastAPI routes and use `curl -v` to compare the response headers from both servers. Notice what FastAPI adds automatically.

```

from fastapi import FastAPI

app = FastAPI()

@app.get('/')
def root(): return {'message': 'Welcome'}

@app.get('/health')
def health(): return {'status': 'ok'}

@app.post('/echo')
def echo(): return {'echo': 'received'}

```

week-01/day02_fastapi.py — run with: `uvicorn day02_fastapi:app --reload`

REFLECTION · 10 MINUTES

Answer these questions before moving on.

Question 1

Why is the blank line between headers and body significant? What would break if a server or client missed it?

Question 2

The Host header is mandatory in HTTP/1.1 but was not required in HTTP/1.0. What changed between the two versions that made it necessary?

Question 3

Your manual HTTP server uses a Python dictionary as a routing table. FastAPI uses decorators. What does FastAPI's approach add beyond what your dictionary does?

Question 4

Look at the headers FastAPI sends that your manual server does not. Pick two of them, look them up, and write one sentence about what each one does.

BEFORE TOMORROW

You now know what an HTTP message looks like and how a server parses one. Tomorrow you go deeper into the semantics of HTTP — what the different methods actually mean, what status codes communicate, and why those semantics matter for designing APIs that other developers can use without documentation.

Commit before you close your laptop

```
git add .
```

```
git commit -m "Day 2: HTTP request/response structure, manual HTTP server"
```

Push to your remote repository on GitHub.

Day 3

HTTP Methods and Status Codes

Week One · How the Internet Talks to Your Code

HTTP gives you more than a way to send data. It gives you a vocabulary for describing what you intend to do with that data. The method — GET, POST, PUT, PATCH, DELETE — communicates intent. The status code in the response communicates outcome. When both are used correctly, an API becomes self-documenting. When they are misused, the API becomes a source of confusion for every developer who works with it.

Today you will understand the semantics of each method, learn the most important status codes in each family, and build a server that uses all of them correctly. Semantics might sound abstract — by the end of today's practical you will see exactly why they are not.

THEORY · 20 MINUTES

HTTP methods and what they mean

Each HTTP method has a defined meaning. These meanings are not enforced by the protocol itself — nothing stops you from writing a GET endpoint that deletes data. But violating the semantics causes real problems: clients cache things they should not, browsers retry requests they should not, and developers reading your API are confused about what is safe to call.

Term	Meaning
<i>GET</i>	Retrieve a resource. Must not change any data. Safe to call multiple times — idempotent and cacheable.
<i>POST</i>	Create a new resource or trigger an action. Not idempotent — calling it twice creates two things.
<i>PUT</i>	Replace a resource entirely. Idempotent — calling it twice with the same data leaves the system in the same state.
<i>PATCH</i>	Partially update a resource. Only the fields sent are changed. Not necessarily idempotent.
<i>DELETE</i>	Remove a resource. Idempotent — deleting something already gone should not return an error.
<i>HEAD</i>	Same as GET but the server returns only headers, not the body. Used to check if a resource exists.
<i>OPTIONS</i>	Ask the server what methods are supported for a path. Used by browsers for CORS preflight checks.

Idempotency — why it matters

Idempotent means: calling the operation once produces the same result as calling it ten times. GET, PUT, and DELETE are idempotent. POST and PATCH are not.

This matters because networks are unreliable. If a client sends a request and never receives a response — the connection dropped — it does not know whether the server processed the request or not. For an idempotent operation, the safe recovery is to retry. For a non-idempotent operation like POST, retrying might create duplicate records. This is why payment systems invest in idempotency keys — a way to make POST operations safe to retry.

Status code families

Term	Meaning
<i>1xx — Informational</i>	The request was received and processing is continuing. Rarely seen in practice.
<i>2xx — Success</i>	The request was received, understood, and accepted.

<i>3xx — Redirection</i>	Further action is needed. The client should follow the Location header.
<i>4xx — Client Error</i>	The request contains an error the client made. The client should not retry unchanged.
<i>5xx — Server Error</i>	The server failed to fulfil a valid request. The client may retry.

The specific codes you must know

Term	Meaning
<i>200 OK</i>	Standard success for GET, PUT, PATCH, DELETE.
<i>201 Created</i>	A new resource was created. Use after POST. Include a Location header.
<i>202 Accepted</i>	Accepted for processing but not yet complete. Use for background jobs.
<i>204 No Content</i>	Success but nothing to return. Common after DELETE.
<i>400 Bad Request</i>	The request is malformed — missing fields, wrong types, invalid values.
<i>401 Unauthorized</i>	The client is not authenticated. Send credentials.
<i>403 Forbidden</i>	Authenticated but not authorised to access this resource.
<i>404 Not Found</i>	The resource does not exist.
<i>409 Conflict</i>	The request conflicts with current state — duplicate record, version mismatch.
<i>422 Unprocessable Entity</i>	Syntactically valid but semantically wrong. FastAPI uses this for validation errors.
<i>429 Too Many Requests</i>	The client has sent too many requests. Used for rate limiting.
<i>500 Internal Server Error</i>	Something went wrong on the server. Do not expose internal details.
<i>503 Service Unavailable</i>	The server is temporarily unable to handle the request.

PRACTICAL · 60 MINUTES

You are going to build a FastAPI server that uses methods and status codes correctly — and then deliberately misuse them to see what goes wrong.

1. Build the correctly-designed API (25 minutes)

Create a student resource API that uses every method and status code correctly. The data is in-memory for now — no database yet.

```
from fastapi import FastAPI, HTTPException
from fastapi.responses import JSONResponse
from pydantic import BaseModel
from typing import Optional

app = FastAPI()
students: dict[int, dict] = {}
next_id = 1

class StudentCreate(BaseModel):
    name: str; email: str; course: str

class StudentUpdate(BaseModel):
    name: Optional[str] = None
    email: Optional[str] = None
    course: Optional[str] = None

@app.get('/students')
def list_students(): return list(students.values())

@app.post('/students', status_code=201)
def create_student(data: StudentCreate):
    global next_id
    student = {'id': next_id, **data.model_dump()}
    students[next_id] = student; next_id += 1
    return JSONResponse(201, student, headers={'Location':
f"/students/{student['id']}"})

@app.get('/students/{sid}')
def get_student(sid: int):
    if sid not in students: raise HTTPException(404, 'Not
found')
    return students[sid]

@app.patch('/students/{sid}')
def update_student(sid: int, data: StudentUpdate):
```

```

        if sid not in students: raise HTTPException(404, 'Not
found')
        students[sid].update(data.model_dump(exclude_none=True))
        return students[sid]

@app.delete('/students/{sid}', status_code=204)
def delete_student(sid: int):
    if sid not in students: raise HTTPException(404, 'Not
found')
    del students[sid]

```

week-01/day03_methods.py

2. Test every status code path (20 minutes)

Use curl to trigger each status code deliberately. Do not stop until you have seen 200, 201, 204, 404, and 422 in actual responses.

```

# Create — should return 201
curl -s -w "%{http_code}" -X POST
http://localhost:8000/students \
    -H 'Content-Type: application/json' \
    -d '{"name":"Julien","email":"j@uni.edu","course":"CS"}'

# Get — should return 200
curl -s -w "%{http_code}" http://localhost:8000/students/1

# Non-existent — should return 404
curl -s -w "%{http_code}" http://localhost:8000/students/999

# Missing field — should return 422
curl -s -w "%{http_code}" -X POST
http://localhost:8000/students \
    -H 'Content-Type: application/json' -d
'{"name":"Julien"}'

# Delete — should return 204
curl -s -w "%{http_code}" -X DELETE
http://localhost:8000/students/1

```

The -w flag prints only the status code — good for quick verification

3. Observe idempotency (15 minutes)

Create a student, then PATCH the same data to that student three times in a row. Then DELETE the student and call DELETE again on the same ID. What status code do you get the second time?

The second DELETE has no universally agreed answer

HTTP specifies that DELETE should be idempotent — the resource is still gone after both calls.

Whether to return 204 or 404 on the second call is legitimately debated.

Pick a position and be able to defend it. The important thing is that your team makes a consistent choice.

REFLECTION · 10 MINUTES

Answer these questions before committing your work.

Question 1

What is the difference between 401 Unauthorized and 403 Forbidden?

When would each appear in a real application?

Question 2

Why does POST return 201 instead of 200? What information does 201 communicate that 200 does not?

Question 3

FastAPI automatically returns 422 for validation errors. What is the difference between a 400 Bad Request and a 422 Unprocessable Entity?

Question 4

Your DELETE endpoint returns 404 when the student does not exist. Some designs return 204 regardless. What are the tradeoffs of each approach?

BEFORE TOMORROW

You now understand HTTP methods and status codes at the semantic level. Tomorrow you take the API you have started and design it properly — resource modelling, URL conventions, versioning, and pagination. The goal is an API that someone could use correctly without reading any documentation.

Commit before you close your laptop

`git add .`

`git commit -m "Day 3: HTTP methods, status codes, idempotency experiments"`

Push to your remote repository on GitHub.

Day 4

REST API Design

Week One · How the Internet Talks to Your Code

An API is a contract. Every developer who uses it depends on that contract being stable, consistent, and predictable. A well-designed API communicates its own structure. A poorly-designed one requires a phone call every time someone wants to use it.

Today you will learn what REST actually means — not just that it uses HTTP, but what the architectural constraints are and why they produce good APIs. You will design a non-trivial API on paper before writing a single line of code. The design phase is not optional — engineers who skip it spend twice as long coding and three times as long debugging.

THEORY · 20 MINUTES

What REST actually is

REST stands for Representational State Transfer. It is an architectural style, not a protocol. Roy Fielding defined it in his doctoral dissertation in 2000. The two constraints that directly affect your daily work are the uniform interface — use HTTP's built-in verbs and semantics consistently — and statelessness — the server should not remember anything about the client between requests.

Statelessness is why authentication tokens are sent in every request rather than being stored in a session on the server. Each request must contain all the information the server needs to process it.

Resources, not actions

The most common REST design mistake is building URLs around actions rather than resources. Actions belong in the HTTP method, not the URL path.

```
# Wrong — verb-based URLs
POST /createStudent
GET /getStudent?id=42
POST /deleteStudent

# Correct — noun-based resource URLs
POST /students          # create a student
GET /students/42        # get student 42
DELETE /students/42     # delete student 42
```

The method communicates the action. The URL identifies the resource.

URL design conventions

Term	Meaning
<i>Plural nouns</i>	Use /students not /student. Collections are plural. /students/42 is one member of the students collection.
<i>Lowercase, hyphens</i>	Use /loan-applications not /loanApplications. Hyphens separate words in paths.
<i>Nesting for ownership</i>	/courses/5/enrollments means the enrollments that belong to course 5. Keep nesting to two levels.
<i>No trailing slashes</i>	/students and /students/ are technically different URLs. Pick one and stick to it.
<i>Query strings for filtering</i>	Use the path to identify a resource, query strings to filter or sort (/students?year=2&page=2).

Versioning

APIs change. When they change in ways that break existing clients, you need to release the new version without breaking old consumers. URL versioning — `/v1/students`, `/v2/students` — is the most visible and easiest to understand. For this curriculum and your projects, use URL versioning. It is explicit, easy to test in a browser, and easy to route at the infrastructure level.

Pagination

A `/students` endpoint that returns every student in the database is a liability. Every collection endpoint that could return more than a few dozen items must be paginated. The client sends `?page=2&per_page=20` and the server returns 20 items at the right offset, along with metadata about the total count and whether there are more pages.

PRACTICAL · 60 MINUTES

1. Design on paper first (15 minutes)

Before writing any code, design the full API for a Student Management System. The system has students, courses, and enrollments. For each route, define: the method, the full path, query parameters, request body, response body, and status code.

Design hints

Start by listing every noun: Student, Course, Enrollment.

For each noun ask: can it be listed? retrieved by ID? created? updated? deleted?

For relationships: can you list a student's courses? A course's students?

Each question maps to one route.

2. Implement the full API with pagination and versioning (35 minutes)

Implement all your designed routes. Focus on correct design — right methods, right status codes, right URL structure.


```

from fastapi import FastAPI, HTTPException, Query
from pydantic import BaseModel

app = FastAPI(title='Student Management API',
version='1.0.0')

students_db: dict[int, dict] = {}
courses_db: dict[int, dict] = {}
enrollments_db: list[dict] = []
counters = {'students': 1, 'courses': 1}

class StudentIn(BaseModel):
    name: str; email: str; year: int

class CourseIn(BaseModel):
    title: str; code: str; credits: int

@app.get('/v1/students')
def list_students(page: int = Query(1, ge=1),
                  per_page: int = Query(20, ge=1, le=100)):
    all_s = list(students_db.values())
    start = (page - 1) * per_page
    return {'data': all_s[start:start+per_page], 'page':
page,
            'total': len(all_s), 'has_more': start+per_page <
len(all_s)}

@app.post('/v1/students', status_code=201)
def create_student(data: StudentIn):
    sid = counters['students']
    students_db[sid] = {'id': sid, **data.model_dump()}
    counters['students'] += 1
    return students_db[sid]

@app.get('/v1/students/{sid}')
def get_student(sid: int):
    if sid not in students_db: raise HTTPException(404,
'Student not found')
    return students_db[sid]

@app.get('/v1/students/{sid}/courses')
def get_student_courses(sid: int):
    if sid not in students_db: raise HTTPException(404,
'Student not found')

```

```

        cids = [e['course_id'] for e in enrollments_db if
e['student_id'] == sid]
        return [courses_db[c] for c in cids if c in courses_db]

@app.post('/v1/students/{sid}/courses/{cid}',
status_code=201)
def enroll(sid: int, cid: int):
    if sid not in students_db: raise HTTPException(404,
'Student not found')
    if cid not in courses_db: raise HTTPException(404,
'Course not found')
    already = any(e['student_id']==sid and
e['course_id']==cid for e in enrollments_db)
    if already: raise HTTPException(409, 'Already enrolled')
    enrollments_db.append({'student_id': sid, 'course_id':
cid})
    return {'student_id': sid, 'course_id': cid}

```

week-01/day04_rest_design.py — this is your reference API for the rest of Week One

3. The documentation test (10 minutes)

Open <http://localhost:8000/docs>. Ask yourself: could a developer who has never spoken to you use this API correctly from the docs page alone? If not, add Pydantic Field descriptions and FastAPI route descriptions until the answer is yes.

REFLECTION · 10 MINUTES

These questions are about design, not syntax. There is no single right answer — but you should be able to articulate a position.

Question 1

Your enrollment route is POST `/v1/students/{sid}/courses/{cid}`. An alternative is POST `/v1/enrollments` with IDs in the body. What are the tradeoffs between these two approaches?

Question 2

REST is stateless — the server does not remember anything between requests. How does authentication work within this constraint?

Question 3

If you needed to release a breaking change to your `/v1/students` endpoint, how would you do it without breaking existing consumers already using `v1`?

Question 4

Your pagination returns a `has_more` flag. Some APIs return `next_page` and `prev_page` URLs instead. Which approach would you choose for a public API and why?

BEFORE TOMORROW

Your API now has a clean design. Tomorrow you go deeper into one aspect of it — data validation and serialisation. Pydantic does a lot of work for you, but you are going to understand what that work actually is and what happens when things go wrong.

Commit before you close your laptop

```
git add .
```

```
git commit -m "Day 4: REST API design, resource modelling, versioning, pagination"
```

Push to your remote repository on GitHub.

Day 5

JSON and Pydantic

Week One · How the Internet Talks to Your Code

Every time data moves across a network it has to be converted into a format that can travel as bytes. That conversion is serialisation. The reverse — converting bytes back into data your program can work with — is deserialisation. Today you will understand what is happening during that process and why validation is inseparable from it.

Pydantic is the library FastAPI uses for both. By the end of today you will understand what Pydantic is doing internally — not just how to declare a model, but why the model exists, what it checks, and what happens when those checks fail.

THEORY · 20 MINUTES

Why serialisation exists

A Python dictionary is a Python object. It lives in your program's memory as a collection of pointers to other objects. That structure is specific to Python and specific to the current running process. You cannot send it across a network directly, because the machine on the other side does not know what a Python dictionary looks like in memory.

Serialisation converts that structure into a format that is universal — a sequence of bytes that any language and any system can interpret. JSON is the

most common format for web APIs. When you return a dictionary from a FastAPI route, FastAPI serialises it to a JSON string before sending it. When a request arrives with a JSON body, FastAPI deserialises the JSON string back into Python objects before your route handler sees it.

What JSON actually is

JSON stands for JavaScript Object Notation. It is a text format that supports a small set of data types: strings, numbers, booleans, null, arrays, and objects. That is the entire type system.

This limited type system creates problems. JSON has no date type — dates are serialised as ISO 8601 strings like '2024-01-15T09:00:00Z' and the receiving application is responsible for parsing them. JSON has no integer versus float distinction — the number 1 and 1.0 are the same in JSON. JSON has no binary data type — files must be base64-encoded first. Understanding these limitations matters when you design your API.

What Pydantic does

Pydantic is a data validation library. When you define a Pydantic model, you define a schema — a description of what valid data looks like. When data arrives, Pydantic checks it against that schema and either produces a validated Python object or raises a detailed error.

Pydantic does three things that are often conflated: it validates types (is this field actually an integer?), it coerces types where safe (the string '42' can become the integer 42), and it validates values (is this integer greater than zero?). Understanding which of these three things it is doing in any given situation is the key to working with it effectively.

Validation versus serialisation

Validation happens on the way in — data arrives from the outside world and Pydantic checks that it matches your schema. Serialisation happens on the way out — your Python objects are converted to JSON for the response. These are separate operations and Pydantic handles both, but through different mechanisms.

A common confusion is expecting Pydantic to automatically validate data that your code creates internally. It does not — Pydantic validates data that comes in through its constructor. If you directly assign an invalid value to a model field after construction, Pydantic will not catch it unless you have configured it to do so with `model_config = ConfigDict(validate_assignment=True)`.

PRACTICAL · 60 MINUTES

1. Understand what Pydantic actually checks (20 minutes)

Write models and deliberately break them in every way you can think of. The goal is to understand the error messages, not to avoid them.

```
from pydantic import BaseModel, Field, field_validator
from typing import Optional
from datetime import datetime

class Student(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    email: str = Field(pattern=r'^[\w.+-]+@[ \w-]+\. [\w.]+$')
    year: int = Field(ge=1, le=4)
    gpa: float = Field(ge=0.0, le=4.0, default=0.0)
    joined: Optional[datetime] = None

    @field_validator('name')
    @classmethod
    def name_must_be_alpha(cls, v: str) -> str:
        if not all(c.isalpha() or c.isspace() for c in v):
            raise ValueError('Name must contain only letters and spaces')
        return v.strip().title()

# Valid — should work
s1 = Student(name='julien glory', email='j@knust.edu.gh', year=2)
print(s1.model_dump())
print(s1.model_dump_json())

# String year — Pydantic coerces '2' to 2
s2 = Student(name='Test Name', email='t@uni.edu', year='2')
print(type(s2.year)) # int, not str
```

```
# Datetime as string - Pydantic parses ISO 8601 automatically
s3 = Student(name='Test Name', email='t@uni.edu', year=1,
             joined='2024-09-01T08:00:00')
print(type(s3.joined)) # datetime, not str

# Now uncomment these one at a time and read every error:
# Student(name='Test', email='t@uni.edu', year=5) #
# year out of range
# Student(name='Test123', email='t@uni.edu', year=1) #
# name with numbers
# Student(name='T', email='t@uni.edu', year=1) #
# name too short
```

week-01/day05_pydantic_experiments.py

2. Customise validation error responses (20 minutes)

FastAPI's default 422 response is detailed but not always client-friendly.
Override the exception handler.

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError
from pydantic import BaseModel, Field

app = FastAPI()

@app.exception_handler(RequestValidationError)
async def validation_handler(request: Request, exc:
RequestValidationError):
    errors = [{'field': ' ' -> '.join(str(p) for p in
e['loc'])},
               'message': e['msg'], 'type': e['type']} for e
in exc.errors()]
    return JSONResponse(status_code=422, content={'errors':
errors})

class StudentIn(BaseModel):
    name: str = Field(min_length=2)
    email: str
    year: int = Field(ge=1, le=4)

@app.post('/students')
def create(data: StudentIn): return data.model_dump()
```


week-01/day05_validation_errors.py — send invalid data and compare old vs new error format

3. Nested models and enums (20 minutes)

Real APIs have nested data structures and constrained choices. Build a model for a course with an embedded instructor and a department enum.

```
from pydantic import BaseModel, Field
from enum import Enum

class Department(str, Enum):
    CS = 'computer_science'
    ENG = 'engineering'
    BUS = 'business'

class Instructor(BaseModel):
    name: str; email: str; department: Department

class CourseIn(BaseModel):
    title: str = Field(min_length=3)
    code: str = Field(pattern=r'^[A-Z]{2,4}\d{3}$')
    credits: int = Field(ge=1, le=6)
    instructor: Instructor
    tags: list[str] = []

course = CourseIn(**{
    'title': 'Databases', 'code': 'CS301', 'credits': 3,
    'instructor': {'name': 'Dr. Mensah', 'email':
'm@knust.edu.gh',
                    'department': 'computer_science'},
    'tags': ['sql', 'postgresql'],
})
print(course.model_dump_json(indent=2))
```

week-01/day05_nested_models.py

REFLECTION · 10 MINUTES

Work through these before tomorrow.

Question 1

Why can't you send a Python dictionary directly over an HTTP connection? What needs to happen to it first and why?

Question 2

Pydantic coerced the string '2' into the integer 2 in your year field. Is that always desirable? When might you want to reject a string value even if it could be coerced?

Question 3

JSON has no date type. When you serialise a Student with a datetime field to JSON, what does the datetime become? When a client sends that string back, how does Pydantic convert it back?

Question 4

What is the difference between `model_dump()` and `model_dump_json()`? When would you use each?

BEFORE TOMORROW

You now understand data validation and serialisation. Tomorrow you zoom out from individual routes and think about the structure of an entire application — how to organise code so it can grow without becoming a tangle of logic in route handlers.

Commit before you close your laptop

`git add .`

`git commit -m "Day 5: JSON serialisation, Pydantic validation, nested models"`

Push to your remote repository on GitHub.

Day 6

Application Architecture: Layers

Week One · How the Internet Talks to Your Code

Every application you have written so far this week has all of its logic in one place — the route handler. That works for five routes. It stops working when you have fifty. Today you will learn why layered architecture exists, what each layer is responsible for, and how to refactor a monolithic route handler into a structure that can grow, be tested, and be maintained by more than one person.

This is not about following rules for their own sake. Every principle in this chapter exists because someone built a system without it and suffered a specific, concrete consequence. By the end of today you will be able to name those consequences and explain exactly what each layer boundary prevents.

THEORY · 20 MINUTES

What goes wrong without layers

When all your logic lives in route handlers, four specific problems appear as the application grows.

Testing becomes difficult — to test any business logic, you have to make HTTP requests, which means you need a running server, which means your tests are slow and fragile. Logic gets duplicated — when two routes need to

find a student by email, you write the same database query twice, and when you need to change it, you have to find both places.

The code becomes hard to read — a route handler that validates input, queries the database, applies business rules, sends an email, and formats a response is doing too many things. And changing one part breaks another — when your database schema changes, you have to hunt through every route handler to find every query that needs updating.

The three layers and what each one owns

Term	Meaning
<i>Route layer</i>	Handles HTTP concerns only. Receives the request, validates input with Pydantic, calls the service, returns the response with the correct status code. Contains no business logic and no database queries.
<i>Service layer</i>	Contains all business logic. Enforces rules, orchestrates operations, calls the repository for data. Contains no HTTP or framework code.
<i>Repository layer</i>	Handles all data access. Knows how to read and write from the database. Contains SQL queries or ORM calls. Contains no business logic and no HTTP code.

The dependency direction

The route layer depends on the service layer. The service layer depends on the repository layer. The repository layer depends on nothing but the database. This one-directional dependency is what makes each layer independently testable and replaceable.

In FastAPI, dependencies flow through the Depends() system — FastAPI's implementation of Dependency Injection. Rather than creating service and repository objects inside each route function, you declare that a route depends on a service, and FastAPI creates and provides it. In a test, you provide a mock service instead of the real one, and the route function never knows the difference.

```

# ✗ BAD — all logic in the route handler
@app.post('/students')
def create_student(data: StudentIn, db = Depends(get_db)):
    existing = db.execute('SELECT * FROM students WHERE email
= ?', data.email)
    if existing:
        return JSONResponse(status_code=409,
content={'error': 'Email taken'})
    db.execute('INSERT INTO students ...')
    send_welcome_email(data.email) # side effect buried in
route
    return {'message': 'created'}

# ✓ GOOD — route delegates entirely to service
@app.post('/students', status_code=201)
def create_student(data: StudentIn, service: StudentService =
Depends()):
    return service.create(data) # route knows nothing about
how this works

```

The route layer should be so thin you can read its entire job in one line

PRACTICAL · 60 MINUTES

You are going to take the Day 4 Student Management API and refactor it into a layered architecture. The external behaviour — routes, status codes, request and response shapes — will not change. Only the internal structure changes.

1. Create the folder structure (5 minutes)

Organise your project before writing any code.

```

week-01/day06_layered/
  main.py
  routes/
    students.py
  services/
    student_service.py
  repositories/
    student_repo.py
  models/
    schemas.py

```

This structure scales — adding a resource means adding files, not modifying existing ones

2. Build the repository layer (15 minutes)

The repository holds all data access. Nothing else calls the store directly.

```
# repositories/student_repo.py
from typing import Optional

class StudentRepository:
    def __init__(self):
        self._store: dict[int, dict] = {}
        self._next_id = 1

    def find_all(self, skip=0, limit=20) -> list[dict]:
        return list(self._store.values())[skip:skip+limit]

    def find_by_id(self, sid: int) -> Optional[dict]:
        return self._store.get(sid)

    def find_by_email(self, email: str) -> Optional[dict]:
        return next((s for s in self._store.values() if
            s['email']==email), None)

    def save(self, data: dict) -> dict:
        student = {'id': self._next_id, **data}
        self._store[self._next_id] = student
        self._next_id += 1
        return student

    def delete(self, sid: int) -> bool:
        if sid not in self._store: return False
        del self._store[sid]
        return True

    def count(self) -> int:
        return len(self._store)
```

repositories/student_repo.py — no HTTP code, no business logic, only data operations

3. Build the service layer (15 minutes)

The service contains the rules. It uses the repository for data but decides what is valid and what is not.

```
# services/student_service.py
from fastapi import HTTPException
from repositories.student_repo import StudentRepository

class StudentService:
```

```

def __init__(self, repo: StudentRepository):
    self.repo = repo

def list_students(self, page: int, per_page: int) -> dict:
    skip = (page - 1) * per_page
    items = self.repo.find_all(skip=skip, limit=per_page)
    return {'data': items, 'page': page, 'total': self.repo.count()}

def get_student(self, sid: int) -> dict:
    s = self.repo.find_by_id(sid)
    if not s: raise HTTPException(404, 'Student not found')
    return s

def create_student(self, data: dict) -> dict:
    # Business rule: email must be unique
    if self.repo.find_by_email(data['email']):
        raise HTTPException(409, 'Email already registered')
    return self.repo.save(data)

def delete_student(self, sid: int) -> None:
    if not self.repo.delete(sid):
        raise HTTPException(404, 'Student not found')

```

services/student_service.py — knows about HTTP exceptions (for now), but zero SQL, zero route logic

4. Build the route layer (15 minutes)

The route handler should be so thin it almost reads like documentation.

```

# routes/students.py
from fastapi import APIRouter, Depends, Query
from services.student_service import StudentService
from repositories.student_repo import StudentRepository
from models.schemas import StudentIn

router = APIRouter(prefix='/v1/students', tags=['Students'])

def get_service() -> StudentService:
    return StudentService(repo=StudentRepository())

@router.get('')

```



```

def list_students(page: int = Query(1, ge=1),
                  per_page: int = Query(20, ge=1, le=100),
                  svc: StudentService = Depends(get_service)):
    return svc.list_students(page, per_page)

@router.post('/', status_code=201)
def create_student(data: StudentIn, svc: StudentService =
    Depends(get_service)):
    return svc.create_student(data.model_dump())

@router.get('/{sid}')
def get_student(sid: int, svc: StudentService =
    Depends(get_service)):
    return svc.get_student(sid)

@router.delete('/{sid}', status_code=204)
def delete_student(sid: int, svc: StudentService =
    Depends(get_service)):
    svc.delete_student(sid)

```

routes/students.py — each route handler is 2-3 lines. This is the target.

REFLECTION · 10 MINUTES

These questions are about design, not syntax. There is no single right answer — but you should be able to articulate a position.

Question 1

The service layer raises `HTTPException` directly. Some engineers argue that the service layer should not know about HTTP at all — it should raise domain exceptions and the route layer converts them. What are the tradeoffs?

Question 2

You could swap the in-memory `StudentRepository` for a PostgreSQL-backed one on Day 8 without changing the service or route layers. What would you have to change in the repository, and what would stay exactly the same?

Question 3

The `get_service()` function creates a new `StudentRepository` on every request. For an in-memory store, this means a fresh empty store per request. What is the problem with this in production and how would you fix it?

Question 4

If you wanted to add rate limiting, which layer would you add it to and why?

BEFORE TOMORROW

Tomorrow is Week One's summary day. No new concept will be introduced. Before you sleep tonight, re-read the code from Days 1 through 5. Notice how each day built directly on the previous one.

Commit before you close your laptop

`git add .`

`git commit -m "Day 6: layered architecture — routes, services, repositories"`

Push to your remote repository on GitHub.

Week 1 Summary

Consolidation and Reflection

No new concept today. This is a consolidation day — time to review what you built, write your weekly summary, clean up your code, and push a complete, organised repository. The goal is not to cover more ground. It is to make sure the ground you covered is solid.

THE WEEKLY SUMMARY

Write five to ten sentences answering this question: what is the complete journey of a single HTTP request from the moment a client sends it to the moment your FastAPI application responds?

Be specific. Use the vocabulary you learned this week. Mention sockets, ports, the HTTP request line, headers, the blank line separator, the routing layer, the service layer, the repository layer, the response status code, and the serialised JSON body. If you cannot describe any part of that journey clearly, that is the part of the week worth revisiting before Week Two.

Write your summary before reading any further

Attempting to write it from memory is the exercise.

Looking things up defeats the purpose.

Save your summary in week-01/SUMMARY.md in your repository.

REVIEW CHECKLIST

Work through each item. If any answer is no, spend time on it today rather than carrying the gap into Week Two.

6. Can you explain what a socket is and how it relates to a port, without using the word 'connection' in your explanation?
7. Can you write a valid HTTP request as plain text from memory — request line, headers, blank line, body — without looking anything up?
8. Can you name every HTTP method and give a one-sentence description of what each one means semantically?
9. Can you give a concrete example of an idempotent and a non-idempotent operation from your own Day 3 code?
10. Can you explain the difference between 401 and 403 to a non-technical person using a plain analogy?
11. Can you explain what Pydantic does in one sentence that a developer who has never used it would understand?
12. Can you describe the three layers of your Day 6 architecture and give one concrete reason why each layer boundary exists?

CODE REVIEW

Open your repository and look at every file from Days 1 through 6. Ask yourself these questions about each one.

For each file

Do the variable names describe what the variable actually contains?

Is every comment explaining why, not just what the code does?

Are there any functions longer than 20 lines that could be split into two?

Is there any duplicated logic that belongs in a shared function?

Clean up anything that bothers you. Rename variables that are unclear. Remove comments that say the same thing as the code. Split functions that

are doing two things. This is the kind of work that distinguishes engineers who write code from engineers who maintain it.

REFLECTION QUESTIONS

Answer these in writing or in your head — but answer them before moving on.

Looking back at Week One

Which day's concept took the longest to click, and what finally made it make sense?

Which experiment from the practical sections taught you the most — and what did it teach you?

Is there anything from the theory that still feels abstract? If so, what would you need to build to make it concrete?

Looking forward to Week Two

Week Two is entirely about databases — where data actually lives between requests.

Your current API stores data in a Python dictionary that disappears when the server restarts.

What problems does that create? What would need to be true about a storage system that solved those problems?

Write down your answer now, before Day 8. Then compare it to what you learn.

FINAL COMMIT

Before you finish today, your repository should contain:

13. `week-01/day01_server.py` — raw TCP socket server
14. `week-01/day02_http_server.py` — manual HTTP request parser and router

15. week-01/day02_fastapi.py — FastAPI equivalent for comparison
16. week-01/day03_methods.py — full CRUD with correct HTTP methods and status codes
17. week-01/day04_rest_design.py — paginated, versioned REST API
18. week-01/day05_pydantic_experiments.py — Pydantic validation experiments
19. week-01/day05_validation_errors.py — custom error handler
20. week-01/day05_nested_models.py — nested Pydantic models
21. week-01/day06_layered/ — complete layered architecture
22. week-01/SUMMARY.md — your written weekly summary

```
git add .
git commit -m "Week 1 complete: HTTP, REST, Pydantic, layered
architecture"
git push
```

Your final commit for Week One

WEEK TWO PREVIEW

Week Two is about databases. Every system you will ever build needs to persist data — to store it somewhere that survives a server restart, that can be queried efficiently, and that can be updated consistently even when multiple users are changing it at the same time.

You will install PostgreSQL locally and connect to it directly without any ORM or framework. You will write SQL by hand. You will design your first real database schema. And by Day 15, at the end of the week, you will understand why some queries take milliseconds and others take minutes — and exactly how to fix the slow ones.

The database week is the most important week in this curriculum. Take your time. If a concept does not click, stay on it. The weeks that follow all depend on a solid understanding of what happens inside a relational database.